

This is a copy of a chat between Claude and surya. Content may include unverified or unsafe content that do not represent the views of Anthropic. Shared snapshot may contain attachments and data not displayed here. [Report](#)

explain this python concepts 2. Variables, Names and Memory Model
Names
Bindings
References
Objects

Python's memory model is different from languages like C or Java, and understanding it clears up a lot of confusion about how assignment, mutation, and function arguments behave. Here's the breakdown:

Objects

Everything in Python is an **object** — a chunk of memory that holds:

- a value (the actual data)
- a type (int, list, str, etc.)
- an identity (a unique id, basically its memory address)

```
python
```

```
x = 42
```

Here, `42` is an object living somewhere in memory. It has a type (`int`), a value (`42`), and an identity you can check with `id(42)`.

Names

A **name** is just a label — a string like `x` or `my_list` that you use to refer to an object. The name itself isn't a box that stores a value. It's more like a sticky note pointing at an object.

```
python
```

```
x = 42
```

`x` is the name. `42` is the object. The name `x` doesn't "contain" 42 — it points to wherever 42 lives.

Bindings

A **binding** is the act of attaching a name to an object — what happens when you write `name = object`. This is the key mental shift from C-like languages: Python's `=` doesn't copy data into a variable, it creates (or updates) an association between a name and an object.

```
python

x = 42      # binds name x to object 42
x = "hello" # rebinds x to a different object "hello"
            # the int 42 is now unreferenced, eligible for garbage collection
```

Each assignment is a fresh binding, not a mutation of some fixed storage slot.

References

A **reference** is the actual link/pointer from a name (or from inside another object, like a list element) to an object. Multiple names can reference the *same* object:

```
python

a = [1, 2, 3]
b = a      # b is bound to the SAME object a points to
b.append(4)
print(a)    # [1, 2, 3, 4] — because a and b reference the same list
```

No copy happened here. `a` and `b` are two different names referencing one object. This is why mutation through one name is visible through the other.

Compare that to:

```
python

a = [1, 2, 3]
b = a
b = [9, 9, 9] # rebinds b to a NEW object
print(a)      # [1, 2, 3] — a is untouched
```

Here `b = [9,9,9]` doesn't mutate the list — it just makes `b` point somewhere else. `a` still points to the original.

Putting it together

Concept	What it is
Object	The actual data in memory (value + type + id)
Name	A label/identifier in your code
Reference	The link from a name to an object
Binding	The act of creating that name→object link

A useful way to picture it: **objects float in memory, names are tags tied to them with string**. `=` doesn't move data between boxes — it ties a tag onto an object. Multiple tags can be tied to the same object (aliasing), and untying a tag (rebinding or deleting it) doesn't affect the object if other tags still hold it.

This is also why Python distinguishes **mutable** objects (lists, dicts, sets — can be changed in place, so all references see the change) from **immutable** ones (int, str, tuple — "changing" them really means binding the name to a brand new object).

Want me to follow up with how this plays into function arguments (pass-by-object-reference) or `is` vs `==`?

Identity
Equality
Mutability
Immutability
Reference Counting
Garbage Collection

Continuing the memory model deep-dive — these six concepts build directly on objects/names/bindings from before.

Identity

Identity is *which object* something is — its actual spot in memory. Every object has exactly one identity, checked with `id()` and compared with `is`.

```
python
a = [1, 2, 3]
b = [1, 2, 3]
print(a is b)          # False — two different objects, same content
```

```
print(id(a), id(b)) # different numbers

c = a
print(a is c)       # True – c is bound to the SAME object as a
```

`is` answers "are these the same object?" — not "do these look the same?"

Equality

Equality is whether two objects have the *same value*, checked with `==`. This calls the object's `__eq__` method, so it's about content, not location.

```
python

a = [1, 2, 3]
b = [1, 2, 3]
print(a == b) # True – same contents
print(a is b) # False – different objects
```

Rule of thumb: use `==` to compare values, use `is` to check identity (most common legitimate use: `x is None`, since there's only ever one `None` object).

```
python

x = None
if x is None:    # idiomatic
    ...
if x == None:    # works, but not idiomatic, and can misbehave for weird custom
    ...
```

Mutability

A **mutable** object can be changed *in place* — its identity stays the same, but its internal state/value changes. Lists, dicts, sets are mutable.

```
python

a = [1, 2, 3]
print(id(a))    # e.g. 140234...
a.append(4)
print(id(a))    # same id – same object, just changed contents
print(a)        # [1, 2, 3, 4]
```

Because mutation happens in place, every name referencing that object sees the change — this is the aliasing behavior from before.

Immutability

An **immutable** object cannot be changed after creation. `int`, `float`, `str`, `tuple`, `frozenset`, `bool` are immutable. "Modifying" one actually creates a brand new object and rebinds the name to it.

```
python

s = "hello"
print(id(s))      # some id
s += " world"
print(id(s))      # DIFFERENT id – a new string was created
print(s)          # "hello world"
```

```
python

x = 5
print(id(x))
x += 1            # not in-place; creates a new int object 6
print(id(x))      # different id
```

This is why immutable objects are safe to share freely — nobody can mutate them out from under you, since "changing" always means creating something new.

Small ints and short strings are often *cached/interned* by CPython, so you may see `is` return `True` for things like `1 is 1` — but never rely on this; it's an implementation detail, not a language guarantee.

Reference Counting

CPython tracks, for every object, **how many references point to it**. This is the *refcount*. When it hits zero, nobody can reach the object anymore, so CPython frees its memory immediately.

```
python

import sys
a = [1, 2, 3]
print(sys.getrefcount(a)) # 2 (one from 'a', one from the getrefcount call's ow

b = a
print(sys.getrefcount(a)) # 3 – another reference now

del b
print(sys.getrefcount(a)) # back down
```

Rebinding, deleting a name, a variable going out of scope, or removing an item from a container — all decrement the refcount of whatever object it referenced. This is why Python's memory management is mostly automatic and immediate for the common case.

Garbage Collection

Reference counting alone can't catch everything — specifically, **reference cycles**, where objects reference each other but nothing outside reaches either of them:

```
python

import gc

class Node:
    def __init__(self):
        self.other = None

a = Node()
b = Node()
a.other = b
b.other = a    # cycle: a -> b -> a

del a
del b
# Even after del, a and b still reference each other,
# so their refcounts never hit zero through refcounting alone.
```

This is where Python's **garbage collector** (the `gc` module) comes in — it periodically scans for groups of objects that reference each other but are unreachable from anywhere else in the program, and collects them.

```
python

gc.collect()    # manually trigger a cycle-collection pass
```

So Python actually uses **two** memory-reclamation mechanisms together:

1. **Reference counting** — immediate, handles the common non-cyclic case
2. **Cyclic garbage collector** — periodic, catches reference cycles that refcounting misses

Tying it all together

Concept	Question it answers
Identity	Is this <i>literally</i> the same object?
Equality	Does this have the <i>same value</i> ?
Mutability	Can this object's value change without changing identity?
Immutability	Does changing the value require a <i>new</i> object?
Reference counting	How many names/containers point to this object right now?
Garbage collection	How do we clean up objects refcounting can't catch (cycles)?

The throughline: Python objects live independently in memory; names are just references to them; whether mutation is visible through other references depends on mutability; and the runtime reclaims memory once nothing (refcount) or no reachable cycle (GC) points to an object anymore.

Want to go next into how this affects **function arguments** (the "pass by object reference" model) or into `copy` vs `deepcopy`, which is where mutability + references collide most often in practice?

Cyclic GC
Memory Pools
Arenas
PyObject Structure
Small Integer Cache
String Interning

Now we're going under the hood into CPython's actual implementation details — this is where "memory model" stops being abstract and becomes real C structs and allocators.

Cyclic GC

We touched on this briefly — now the mechanics. The cyclic garbage collector exists because reference counting alone can't free objects that reference each other in a cycle, even when nothing outside reaches them.

How it actually works: CPython's GC only tracks *container* objects (lists, dicts, classes, objects with `__dict__`, etc.) — not simple immutables like ints or strings, since those

can't form cycles. It organizes tracked objects into **generations** (0, 1, 2), based on the idea that young objects die young (generational hypothesis):

```
python

import gc

print(gc.get_threshold())    # e.g. (700, 10, 10)
# gen0 collected after 700 allocations - deallocations
# gen1 collected once for every 10 gen0 collections
# gen2 collected once for every 10 gen1 collections
```

New objects start in generation 0. Every time gen0's allocation count crosses its threshold, the collector runs a cycle-detection pass over gen0. Objects that survive a collection get promoted to the next generation, which gets scanned less often — because if something's stuck around a while, it's probably going to keep sticking around.

Detection algorithm, roughly:

1. For each tracked object, copy its refcount into a shadow field.
2. Walk all tracked objects and subtract 1 from that shadow count for every internal reference found (i.e., references coming from *other tracked objects*).
3. Whatever's left with shadow count > 0 is reachable from outside the cycle-candidate set — keep it.
4. Whatever has shadow count 0 is only reachable via other objects in the candidate set — meaning it's part of an unreachable cycle. Collect it.

```
python

gc.collect()          # force a full collection (all generations)
gc.collect(0)         # force only generation 0
```

As of Python 3.13+, there's also been work toward making GC "free-threaded"-friendly and incremental — but the generational/refcount-shadow logic above is the conceptual core that's stayed stable for years.

Memory Pools

CPython doesn't ask the OS for memory every time you create a small object — that'd be brutally slow. Instead it uses **pymalloc**, a custom allocator layered on top of the OS allocator (malloc), specifically tuned for lots of small, short-lived objects (which is exactly Python's workload).

A **pool** is a 4KB chunk of memory, subdivided into equal-sized "blocks." Each pool holds blocks of *one specific size class* (sizes are rounded up to multiples of 8 bytes, e.g.

8, 16, 24, ... up to 512 bytes).

Pool (4KB)

- └─ Block (e.g. 32 bytes) – holds one small object
- └─ Block (32 bytes)
- └─ Block (32 bytes)
- └─ ... etc

When you allocate a small object, pymalloc finds (or creates) a pool matching that size class and hands out a free block from it — much cheaper than a full OS-level malloc call.

Arenas

Pools themselves live inside **arenas** — 1MB chunks of memory requested directly from the OS.

Arena (1MB)

- └─ Pool (4KB) – size class 16 bytes
- └─ Pool (4KB) – size class 32 bytes
- └─ Pool (4KB) – size class 16 bytes
- └─ ... (256 pools per arena)

So the hierarchy is:

OS memory

- └─ Arena (1MB)
 - └─ Pool (4KB)
 - └─ Block (8-512 bytes) ← actual object storage

This three-tier structure (arena → pool → block) is why Python handles small-object churn (creating/destroying tons of small ints, tuples, etc.) so efficiently — it avoids hitting the OS allocator constantly, and arenas that become completely empty can be returned to the OS.

Objects larger than 512 bytes skip pymalloc and go straight to the system allocator (`malloc` / `free`), since the small-object optimization doesn't pay off at that size.

PyObject Structure

Every Python object, at the C level, starts with a common header — `PyObject`:

c

```
typedef struct _object {
    Py_ssize_t ob_refcnt;    // the reference count
    PyObject *ob_type;    // pointer to the type object (e.g. int, list, ...)
} PyObject;
```

That's it at the base — just refcount and type pointer. Every other Python object (int, list, dict, your custom class instance) is this struct *plus* extra fields tacked on. For example, variable-sized objects (str, list, tuple) use `PyVarObject`, which adds a size field:

```
c
typedef struct {
    PyObject ob_base;
    Py_ssize_t ob_size;    // number of items
} PyVarObject;
```

And a concrete type extends further — e.g. a Python `int` (which is arbitrary-precision) is roughly:

```
c
typedef struct {
    PyObject ob_base;
    Py_ssize_t ob_size;
    digit ob_digit[1];    // array of "digits" holding the actual number
} PyLongObject;
```

This is *why* `id()` works and is meaningful — it literally returns the memory address of this struct. And it's why `ob_refcnt` ties directly into reference counting:

`sys.getrefcount()` just reads this field.

Small Integer Cache

CPython pre-creates and caches `int` objects in the range **-5 to 256** at interpreter startup. Any time your code uses an integer in that range, you get a reference to the *same* cached object instead of a new one being allocated.

```
python
a = 100
b = 100
print(a is b)    # True — both point to the cached int 100

a = 1000
```

```
b = 1000
print(a is b)      # False (usually) – outside the cache range, separate objects
```

This is purely a performance/memory optimization (small ints come up constantly — loop counters, indices, etc.) and is a CPython implementation detail, not a language guarantee. Never write code that depends on `is` working for integers — always use `==` for value comparison.

```
python

# DON'T do this:
if x is 5:      # fragile – relies on implementation detail
    ...

# DO this:
if x == 5:
    ...
```

String Interning

Similarly, CPython **interns** certain strings — meaning it keeps a single shared object for a string value instead of creating duplicates. This mainly happens automatically for:

- String literals that look like identifiers (letters, digits, underscores) — e.g. variable names, since the interpreter's parser interns them as part of compiling code.
- Strings explicitly interned via `sys.intern()`.

```
python

a = "hello"
b = "hello"
print(a is b)      # True – interned, same object (typical for identifier-like lit

a = "hello world!"
b = "hello world!"
print(a is b)      # often False – strings with spaces/punctuation aren't auto-int

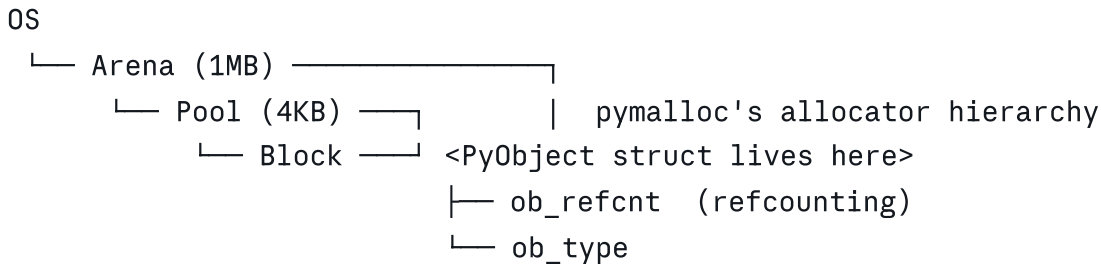
import sys
a = sys.intern("hello world!")
b = sys.intern("hello world!")
print(a is b)      # True – now forced to share the same object
```

Interning matters for performance: dict key lookups and string comparisons can short-circuit to a pointer comparison when both strings are interned, before falling back to

character-by-character comparison. CPython leans on this heavily for attribute names and dict keys, since those are compared constantly.

Same caveat as ints: **never rely on `is` for string equality** in your own code — interning behavior isn't part of the language spec and can change between versions/implementations.

Pulling the whole picture together



- **Reference counting** handles most deallocation, reading/writing `ob_refcnt` directly inside the `PyObject` struct.
- **Cyclic GC** sweeps in periodically for what refcounting can't catch — reference cycles among tracked container objects.
- **pymalloc (arenas → pools → blocks)** is the actual memory-supplying machinery underneath all of this, optimized for Python's many-small-objects-churning-constantly pattern.
- **Small int cache & string interning** are targeted optimizations that exploit immutability — since these objects can never change value, it's safe to silently share one object across many names.

This is genuinely CPython-specific — other implementations (PyPy, MicroPython, Jython) are free to manage memory completely differently while still being "Python." Want to go next into how all this shapes **function argument passing** (the actual mechanics of "pass-by-object-reference"), or into `copy` / `deepcopy` semantics, which is where mutability + this whole memory model collide most practically in everyday code?

